Ders 2 Genişletilmiş Notu: Öğrenci Sunumları: Mikroservis Mimarisine Giriş, Saga ile İşlem Yönetimi ve Servislere Ayrıştırma

Ders 2 Genişletilmiş Notu: Öğrenci Sunumları: Mikroservis Mimarisine Giriş, Saga ile İşlem Yönetimi ve Servislere Ayrıştırma

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Mikroservis mimarisi nedir
- 2. Bağımsızlık, esneklik, avantajlar ve zorluklar
- 3. Yazılım mimarisi perspektifleri: 4+1, katmanlı ve altıgen mimari
- 4. FTGO örneğiyle mikroservislerin pratik kullanımı
- 5. Transaction yönetimi ve ACID
- 6. Monolitte transaction: Spring ve @Transactional
- 7. Geleneksel çözüm: X/Open XA Two-Phase Commit
- 8. Modern çözüm: Saga tasarım deseni
- 9. Servislere ayrıştırma süreci ve sistem operasyonları
- 10. Yüksek seviyeli alan modeli
- 11. İş yeteneklerine göre ayrıştırma
- 12. Alt alan (subdomain) desenine göre ayrıştırma
- 13. Ayrıştırma yönergeleri ve engeller
- 14. Servis API'lerinin tasarımı ve sonuç

Bu ders tamamen öğrenci sunumlarından oluşur. Üç öğrenci, dersin mikroservis kitabından aldıkları bölümleri anlatır. Bütün örnekler kitabın FTGO adlı yemek siparişi ve teslimatı uygulaması üzerinden ilerler. Birinci sunum mikroservis mimarisine giriş yapar (Bölüm 1-4). İkinci sunum mikroservislerde transaction yönetimini ve Saga desenini anlatır (Bölüm 5-8). Üçüncü sunum, kitabın ikinci bölümüne dayanarak bir uygulamanın servislere nasıl ayrıştırılacağını anlatır (Bölüm 9-14). Sunan kişiler bu notta "bir öğrenci" diye anılır.

> **Ek bilgi (derste anlatılmadı):** FTGO örneği ve sunumlarda kullanılan şekiller büyük olasılıkla Chris Richardson'ın *Microservices Patterns* kitabından alınmıştır. Bu kitapta servislere ayrıştırma ikinci bölümün, Saga ile transaction yönetimi dördüncü bölümün konusudur.

1. Mikroservis mimarisi nedir

Birinci sunumu yapan öğrenci, sunumu giriş, mimarinin temelleri, yazılım mimarisi perspektifleri, pratik kullanım ve sonuç diye beş kısma ayırır.

Mikroservis mimarisi, bir uygulamanın işlevsel parçalara bölünmesi esasına dayanır. Bu parçalara mikroservis denir. Her mikroservis bağımsız olarak geliştirilebilir ve dağıtılabilir (deploy edilebilir). Bu yaklaşım büyük ve karmaşık sistemleri daha yönetilebilir ve esnek hale getirir.

Neden mikroservise ihtiyaç duyulduğunu anlamak için geleneksel monolitik mimariye bakılır. Monolitik mimaride bütün uygulama tek bir yapıdır. Herhangi bir değişiklikte bütün sistemin yeniden inşa edilmesi ve yeniden dağıtılması gerekir. Mikroservislerde ise her hizmet bağımsızdır. Bu bağımsızlık ekiplerin daha hızlı ve verimli çalışmasını sağlar. Öğrenciye göre mikroservisler daha hızlı geliştirme, kolay bakım ve yüksek ölçeklenebilirlik sunduğu için günümüz projelerinde sıkça tercih edilir.

2. Bağımsızlık, esneklik, avantajlar ve zorluklar

2.1 Bağımsızlık

Mikroservis mimarisinde her servis kendi işlevini ve kendi veri tabanını yönetir. Bu sayede bir servis, diğerlerinden etkilenmeden geliştirilebilir ve dağıtılabilir.

Öğrencinin örneği üç servisten oluşur: sipariş yönetimi, bildirim ve ödeme. Sipariş yönetimi servisinde yapılan bir güncelleme bildirim ya da ödeme servisini etkilemez. Bu, mikroservislerin önemli bir avantajıdır.

2.2 Esneklik

Esneklik, gerektiğinde yalnız belirli bir servisin ölçeklenebilmesi ya da güncellenebilmesidir. Aynı örnekte ödeme servisi diğerlerinden bağımsız ölçeklenebilir. Her servisin konfigürasyonu ayrı tutulabilir. Ayrıca her servis farklı bir teknoloji yığınıyla geliştirilebilir.

Öğrenci bunu bir izleme örneğiyle somutlaştırır. Ödeme servisine Dynatrace ya da OpenTelemetry gibi izleme entegrasyonları eklenebilir. Bu entegrasyonlar performansı biraz düşürür. Ama para geçen bir yerde işlemlerin izini (trace) sürebilmek daha değerlidir. Bu karar yalnız ödeme servisini ilgilendirir, diğer servislere yansımaz. Böylece ekipler projeye uygun teknolojiyi servis bazında seçebilir.

2.3 Avantajlar ve zorluklar

Slayttaki özet şöyledir (ekrandan, 00:10:00):

- **Avantajlar:** hızlı geliştirme ve dağıtım; kolay bakım ve daha iyi ölçeklenebilirlik; ekipler arasında bağımsız çalışma imkânı.
- **Zorluklar:** karmaşık yapının yönetimi; veri tutarlılığı sorunları; izleme ve hata ayıklama zorlukları.

Avantajlar tarafında öğrenci paralel çalışmayı vurgular. Bir ekip A servisi üzerinde çalışırken başka bir ekip B servisi üzerinde çalışabilir. Bu, geliştirme süresini önemli ölçüde kısaltır ve büyük organizasyonlarda üretkenliği artırır.

Zorluklar tarafında üç konu öne çıkar. Birincisi, farklı servislerin koordinasyonu karmaşıktır. İkincisi, her servisin kendi veri tabanına sahip olması sistem genelinde veri tutarlılığını sağlamayı güçleştirir. Üçüncüsü, hata ayıklama ve sistemin genel izlenebilirliği dikkatle ele alınması gereken konulardır. Öğrencinin vardığı sonuç, mikroservislerin doğru uygulandığında yazılım geliştirmeye büyük değer kattığıdır.

3. Yazılım mimarisi perspektifleri: 4+1, katmanlı ve altıgen mimari

3.1 4+1 görünüm modeli

Yazılım mimarisinde sıkça kullanılan 4+1 görünüm modeli, yazılımı farklı perspektiflerden değerlendirmeyi sağlar. Dört görünüm ve bunları açıklayan bir “+1” senaryo kümesi vardır. Slayttaki tanımlar şöyledir (ekrandan, 00:11:00):

- **Mantıksal görünüm (logical view):** Yazılımcıların oluşturduğu sınıflar ve paketler ile aralarındaki ilişkiler. Yazılımın geliştirme sırasındaki yapısını anlamak için kullanılır.
- **Uygulama görünümü (implementation view):** Derleme sisteminin ürettiği modüller (JAR dosyaları) ve bileşenler (WAR dosyaları ya da çalıştırılabilir dosyalar) ile aralarındaki bağımlılıklar. Kodun dağıtım öncesi nasıl paketlendiğini gösterir.

- **Süreç görünümü (process view):** Çalışma zamanındaki bileşenler (süreçler) ve süreçler arası iletişim.
- **Dağıtım görünümü (deployment view):** Süreçlerin fiziksel ya da sanal makinelere nasıl dağıtıldığı; ilişki türü ağ bağlantısıdır.
- **Senaryolar (+1):** Görünümleri "canlandıran" ve aralarındaki iş birliğini açıklayan senaryolar. Yazılımın işlevselliğinin bir bütün olarak anlaşılmasını sağlar.

3.2 Katmanlı mimari

Katmanlı mimaride yazılım belirli sorumluluklara göre katmanlara ayrılır. Genellikle üç katman vardır (ekrandan, 00:12:20):

- **Sunum katmanı (Presentation Layer):** Kullanıcı arayüzünü ya da dış API'leri içerir (giriş/çıkış işlemleri).
- **İş mantığı katmanı (Business Layer):** Uygulamanın kurallarını ve iş mantığını içerir, veriyi işler.
- **Veri katmanı (Data Layer):** Veri tabanıyla etkileşimi sağlar (CRUD).

Bu yapı basit ve anlaşılırdır, geliştiriciler arasında iş bölümüne de olanak tanır. Ama katmanlar arasında sıkı bağımlılıklar vardır. Öğrencinin örneğinde veri katmanında veri tabanına dair yapılan bir değişiklik, bu bağımlılıklar yüzünden bütün sistemi etkileyebilir. Her katmanın bir diğerine bağlı olması test etmeyi de zorlaştırır.

3.3 Altıgen (hexagonal) mimari

Altıgen mimaride iş mantığı merkezdedir ve adaptörlerle çevrilidir. Adaptörler iş mantığını dış sistemlerden ayırır (ekrandan, 00:13:05).

- **Inbound (gelen) adaptörler:** API'lerden ve kullanıcıdan gelen talepleri iş mantığına iletir. Şekilde tarayıcıdan gelen isteği karşılayan bir controller sınıfı ile mesaj aracısından mesaj okuyan bir mesaj tüketicisi (message consumer) bu türdendir. Bunlar iş mantığına bir inbound port (şekilde "Foo service") üzerinden ulaşır.
- **Outbound (giden) adaptörler:** İş mantığından gelen talepleri dış sistemlere iletir. Şekilde veri tabanına erişen DAO ile mesaj aracısına mesaj gönderen mesaj üreticisi (message producer) bu türdendir. İş mantığı bunlara outbound portlar (repository arayüzü, mesajlaşma arayüzü) üzerinden bağlanır.

Altıgen mimarinin en büyük avantajı, iş mantığı ile diğer bileşenlerin birbirinden tamamen bağımsız hale gelmesidir. Bu bağımsızlık test etmeyi kolaylaştırır ve modülerliği artırır. Dezavantajı, katmanlı mimariye göre daha karmaşık bir başlangıç tasarımı gerektirmesi ve kurulum maliyetinin daha yüksek olabilmesidir.

Öğrenci iki mimarinin de kendine özgü avantajları ve kullanım durumları olduğunu, ama günümüz mikroservislerinde genellikle altıgen mimarinin tercih edildiğini söyler.

4. FTGO örneğiyle mikroservislerin pratik kullanımı

Mikroservislerin nasıl organize edildiğini görmek için kitabın FTGO uygulaması kullanılır. FTGO bir restoran sipariş yönetimi senaryosudur. Uygulama sipariş, restoran, mutfak, teslimat, muhasebe ve bildirim gibi bağımsız servislerden oluşur (ekrandan, 00:14:10).

Slayttaki şekilde üç tür kullanıcı vardır: kurye (courier), tüketici (consumer) ve restoran. Kurye ve tüketici mobil uygulamalarla, restoran ise bir web arayüzüyle (Restaurant Web UI) sisteme bağlanır. Mobil uygulamalardan gelen istekleri API Gateway karşılar ve ilgili servise yönlendirir. Order, Restaurant, Kitchen, Delivery, Accounting ve Notification servislerinin her birinin kendi REST API'si vardır. Şekildeki iki not önemlidir: "servislerin API'leri vardır" ve "bir servisin verisi kendisine özeldir". Ayrıca servisler iş yeteneklerine ya da DDD alt alanlarına karşılık gelir. Muhasebe servisi ödeme için bir Stripe adaptörü, bildirim servisi ise Twilio ve Amazon SES adaptörleri kullanır.

Servisler arası iletişim REST API'leri ya da asenkron mesajlaşma ile sağlanır. Bu yapı her servisin bağımsız geliştirilip dağıtılmasını mümkün kılar. Örneğin yalnız sipariş servisi güncellendiğinde diğer servisler kesintiye uğramaz.

Mikroservisler özellikle hızlı geliştirme ve bağımsız ekip çalışması gereken durumlarda öne çıkar. Paralel ekiplerin her biri farklı bir servisten sorumlu olur. Çok büyük ölçekli uygulamalarda ölçeklenebilirlik avantajı belirginleşir. Öğrencinin örneklerinden biri büyük bir e-ticaret sitesidir: yoğun sipariş dönemlerinde yalnız sipariş servisini ölçeklemek mümkündür. Diğeri video platformlarıdır: içerik akışını ayrı servisler üzerinden yöneterek yüksek kullanıcı yükü karşılanır.

4.1 Sonuç: yönetim tarafı

Sunumun sonunda öğrenci mikroservislerin bir de yönetim tarafı olduğunu vurgular. Her servisin ayrı geliştirilip ayrı dağıtılması DevOps işlerini, özellikle başlangıçta, oldukça artırır. Her servis için ayrı dağıtım süreci, ayrı pipeline ve ayrı konfigürasyon yönetimi gerekir. Bu da sistemin karmaşıklığını artırır. Bu yüzden mikroservis mimarisi uygulanırken DevOps ekipleriyle sıkı iş birliği yapılmalı ve teknik altyapı buna uygun hazırlanmalıdır.

Hoca sunumu, giriş bölümünü güzel özetleyerek mikroservis mimarisini aktaran bir sunum olarak değerlendirir (00:17:37).

5. Transaction yönetimi ve ACID

5.1 Problem

İkinci sunumu yapan öğrencinin konusu, kitaptaki “Transaction Management with Saga” bölümüdür: mikroservis mimarisinde transaction yönetiminin Saga adlı tasarım deseniyle nasıl yapıldığı.

Öğrenci transaction yönetimini bir uygulamada veri yapısında yapılan değişikliklerin yönetimi olarak tanımlar. Örnek olarak bir yemek siparişi uygulamasını alır. “Sipariş et” düğmesine basıldığında veri tabanında birçok tablo değişir: uygulamadaki cüzdandan sipariş tutarı kadar para düşülür, yeni bir sipariş kaydı oluşturulur, karmaşık işlemlerde siparişin takibi için kuryenin konumu güncellenir. Birden fazla tablo aynı anda yazma işlemine maruz kalır.

Monolitik ve basit projelerde bu iş görece kolaydır, çünkü veri tabanı tek bir uygulamanın içindedir. Mikroservislerde ise farklı servislerin veri tabanları farklıdır. Bütün işlemler başarılı olursa sorun yoktur. Ama arada bir işlem koparsa, bir servis ayağa kalkamazsa ya da kodda bir adım başarısız olursa veri nasıl korunacaktır? Veri tabanındaki veriyi işlemden önceki haline döndürmeye **rollback** denir. Öğrenciye göre işin zor kısmı budur: rollback, kullanıcı mağdur olmayacak biçimde nasıl yapılmalıdır?

Sunumun akışı şöyledir: önce monolitik yapılarda transaction’ın nasıl kontrol edildiği ve bunun mikroserviste neden verimli olmadığı, sonra mikroserviste kullanılan geleneksel çözüm ve dezavantajları, en son da modern çözüm olan Saga.

5.2 ACID özellikleri

Bir transaction’ın doğru yapılması için ACID protokolündeki dört özelliğin korunması gerekir (ekrandan, 00:30:00):

- **Atomicity (atomiklik):** İşlemler bir bütün olarak ya yapılır ya da hiç yapılmaz. Rollback bu ilkenin temelidir. Örneğin kredi kartıyla sipariş verildiğinde istek birkaç aşamadan geçer ve veri tabanında değişiklik yapar. Kartın süresi geçmişse işlem başarısız olur; o ana kadar yapılmış değişiklikler geri alınmalıdır.
- **Consistency (tutarlılık):** Veri tabanı işlem öncesinde ve sonrasında tutarlı bir durumda kalır. İşlem başarılı olsa da rollback yapılsa da veri tabanı, kullanıcıyı mağdur bırakacak biçimde bozulmamalıdır.
- **Isolation (yalıtım):** Bir işlem diğer işlemlerden etkilenmez.
- **Durability (dayanıklılık):** İşlem başarıyla tamamlandığında sonuç kalıcı olarak saklanır.

Öğrenci yalıtım konusunda bir ayrıma dikkat çeker. Mikroservis mimarisinde farklı veri tabanları birbirinden etkilendiği için Saga ile transaction yönetimi yapılırken bu özellik aynı biçimde sağlanmaz; ortaya çıkan anormallikleri tespit etmenin Saga’da farklı yöntemleri vardır. Öğrenci bu konunun kendi bölümünün kapsamını aştığını söyleyerek ayrıntıya girmez.

Ek bilgi (derste anlatılmadı): Saga'lar ACID'in "I" harfini, yani yalıtımı sağlamaz; bu yüzden kaynaklarda saga'lar için bazen "ACD" denir. Bir saga'nın ara adımlarda yaptığı değişiklikler, saga bitmeden başka işlemlere görünür olur. Kitapta büyük olasılıkla bu yalıtım eksikliğinin yol açtığı anormallikleri azaltmak için ayrı karşı önlemler (countermeasures) anlatılır.

6. Monolitte transaction: Spring ve @Transactional

Monolitik yapılarda transaction yönetimi genellikle tek bir veri tabanında yapıldığı için oldukça basittir. Spring gibi framework'ler sayesinde ACID özelliklerine uygun, güvenilir ve hatalara dayanıklı bir sistem kolayca kurulabilir (ekrandan, 00:31:10). Slaytta Spring Boot ile yazılmış şu kod parçası gösterilir:

```
@Service
public class BankService {

    @Autowired
    private AccountRepository accountRepository;

    @Transactional
    public void transferMoney(Long fromAccountId, Long toAccountId,
        Double amount) {
        // Gönderici hesabından para düşülüyor
        ...
    }
}
```

Öğrenci kodu şöyle açıklar. Monolitik yapıda "servis" denirken ayrı bir portta çalışan başka bir uygulama değil, aynı uygulamadaki bir sınıf kastedilir. `transferMoney` metodu çalışırken bazı doğrulamalar yapılır. Doğrulamadan geçen istek başka bir servis sınıfına, oradan da repository üzerinden veri tabanına gider. Örneğin hesabın ya da banka kartının yetkilendirildiği bir metot başka bir servis sınıfındaysa ve orada bir hata çıkarsa ya da kart bilgilerinde sorun varsa, işlem yapılmamalıdır. O sınıfa gelmeden önce bazı satırlar veri tabanında değişiklik yapmışsa, `@Transactional` anotasyonu bu değişikliklerin geri alınmasını sağlar.

Bunun bu kadar kolay olmasının nedeni bütün tabloların tek bir veri tabanında olmasıdır. Anotasyon, veri tabanına tek bir rollback komutu göndererek işi bitirir. Mikroserviste veri tabanları farklı uygulamalarda olduğunda bu yeterli değildir: her veri tabanında ayrı bir transaction yönetimi gerekir ve işlem birden fazla servisi etkiliyorsa tek veri tabanında rollback yapmak yetmez.

7. Geleneksel çözüm: X/Open XA Two-Phase Commit

İlk akla gelen çözüm, her veri tabanında ayrı ayrı rollback yapmaktır. Ama bu farklı veri tabanlarında yapılırken uygulamanın bunu nasıl yöneteceği sorusu ortaya çıkar. Bunun geleneksel cevabı X/Open XA Two-Phase Commit (iki aşamalı commit) desenidir (ekrandan, 00:33:30):

- İki aşamalı commit ile işlem bütün veri tabanları ve servisler üzerinde senkronize edilir.
- Temel amaç, bütün katılımcıların bir işlemi ya tamamen başarıyla tamamlaması (**commit**) ya da tamamen geri alması (**rollback**) sağlamaktır.
- Bu sayede işlem sırasında tutarlılık korunur.

Bütün servisler işlemi başarıyla tamamlarsa uygulama commit emrini gönderir ve veri tabanları kaydedilir. Bir adım başarısız olursa, bütün veri tabanlarının ayakta olup olmadığı denetlenir ve hepsinde rollback yapılır. Öğrencinin araştırmasına göre protokolde bir hazırlık aşaması vardır: önce bütün servislerin ayakta ve commit'e hazır olup olmadığı kontrol edilir, sonra karar uygulanır.

Dezavantajlar şunlardır:

- **Senkron çalışır.** Commit ya da rollback'in tamamlanabilmesi için bütün servislerin aynı anda ayakta olması gerekir. Veri tabanını sağlıklı biçimde günceller, ama senkron yapısı yüzünden yavaştır; bu da tasarımcıları daha hızlı bir çözüm aramaya yöneltmiştir.

- **NoSQL desteği yoktur.** Her zaman SQL veri tabanı kullanılmaz; MongoDB gibi doküman tabanlı NoSQL veri tabanları XA two-phase commit ile çalışamaz.
- **Modern mesaj aracıları desteklemez.** Kafka ve onun daha basit bir alternatifi olarak anılan RabbitMQ bu protokolü desteklemez. Bu yüzden günümüz uygulamalarında kullanımı verimli değildir.
- **Erişilebilirlik sorunu.** Bir servis ayakta değilse işlem yapılamaz ve gecikme oluşur.

8. Modern çözüm: Saga tasarım deseni

8.1 Temel fikir

Bu sorunlar yüzünden yazılım tasarımcıları Saga desenini geliştirmiştir. Slayttaki tanım iki maddedir (ekrandan, 00:36:50):

- İşlemi adımlara bölerek ve yerel compensating transaction'lar (geri alma işlemleri) tanımlayarak bağımsız bir çözüm sağlar.
- Asenkron mesajlaşma ile verilerin zamanla tutarlı hale gelmesi hedeflenir.

Slayttaki şekilde bir saga'nın tek bir iş sürecini temsil ettiği yazılıdır. Örnek bir seyahat ayarlama sürecidir: toplantı planla, uçuş ayır, otel ayır, ulaşım ayır. Otel ayırma adımı başarısız olunca önceki adımların iptal (cancel) işlemleri geriye doğru çalışır.

Mekanizma şöyle işler. Bir işlem, servis sayısı kadar yerel transaction'a bölünür; dört servis varsa dördünde ayrı yerel transaction'lar çalışır. Bir serviste biten yerel transaction, bir mesaj yayınlayarak sıradakini otomatik olarak tetikler. Bu sayede bir servis geçici olarak durmuş olsa bile mesaj aracısı, o servis ayağa kalkana kadar mesajı tutmaya devam eder. Bir adım başarısız olursa önceki işlemler telafi edilir ve sistem tutarlı hale getirilir.

Telafi işlemleri önceden kaydedilir: her adım için "bu adım başarısız olursa geri almak için ne yapılacak" sorusunun cevabı baştan tanımlanır. Öğrenci önemli bir ayrıntıyı vurgular: geri alma yalnız veri tabanına yazan adımlar için gerekir. Yalnız okuma yapan bir adımın telafisine gerek yoktur.

8.2 T ve C şeması

Slayttaki genel şema şudur (ekrandan, 00:39:00):

```
Saga: T1 ... Tn | T(n+1) FAILS | Cn ... C1

T1...Tn tarafından yapılan değişiklikler commit edilmiştir.
Compensating transaction'lar (Cn...C1) T1...Tn'nin değişikliklerini geri alır.
```

$T_1, \ldots, T_n$ adımları başarıyla commit edilmiş olsun ve T_{n+1} başarısız olsun. Bu durumda telafi işlemleri ters sırayla çalışır: önce C_n, en son C_1. Her C_i, T_i'nin yaptığını geri alır. Öğrencinin para örneği: ilk adımda hesaptan 10 dolar düşülmüşse ve daha sonraki bir adım başarısız olursa, önce aradaki adımlar geri alınır, en sonunda 10 dolar hesaba geri eklenir.

Öğrencinin sözlü verdiği ara örnekte adımlar şöyledir: kartın doğrulanması, banka hesabından paranın düşülmesi, paranın şirketin bütçesine eklenmesi ya da kuryeye mesajın ulaştırılması. Ortadaki servis başarısız olursa, öncesinde yazma yapmış adımlar geri döndürülür.

8.3 FTGO "sipariş oluştur" sagası

Kitaptaki örnekte dört servis vardır: Order Service, Consumer Service, Kitchen Service ve Accounting Service. Slayttaki şekil saga'yı altı yerel transaction olarak gösterir (ekrandan, 00:40:00): Txn:1 Create order (Order), Txn:2 Verify consumer (Consumer), Txn:3 Create ticket (Kitchen), Txn:4 Authorize card (Accounting), Txn:5 Approve ticket (Kitchen), Txn:6 Approve order (Order).

Başarılı senaryodaki adımlar (ekrandan, 00:40:45):

```
1 Order Service      - Order'ı APPROVAL_PENDING durumunda oluştur.
2 Consumer Service   - Tüketicinin sipariş verebildiğini doğrula.
3 Kitchen Service    - Sipariş ayrıntılarını doğrula ve Ticket'ı CREATE_PENDING
```

```
                        durumunda oluştur.
4 Accounting Service - Tüketicinin kredi kartını onayla.
5 Kitchen Service    - Ticket'ın durumunu AWAITING_ACCEPTANCE yap.
6 Order Service      - Order'ın durumunu APPROVED yap.
```

Kart onayının başarısız olduğu senaryo (ekrandan, 00:42:15):

```
1 Order Service      - Order'ı APPROVAL_PENDING durumunda oluştur.
2 Consumer Service   - Tüketicinin sipariş verebildiğini doğrula.
3 Kitchen Service    - Sipariş ayrıntılarını doğrula ve Ticket'ı CREATE_PENDING
                       durumunda oluştur.
4 Accounting Service - Tüketicinin kredi kartını onayla; bu adım başarısız olur.
5 Kitchen Service    - Ticket'ın durumunu CREATE_REJECTED yap.
6 Order Service      - Order'ın durumunu REJECTED yap.
```

İki senaryo karşılaştırıldığında telafinin nasıl çalıştığı görülür. Dördüncü adım başarısız olunca beşinci adımda bileti onaylamak yerine reddeden (CREATE_REJECTED) işlem, altıncı adımda siparişi onaylamak yerine reddeden (REJECTED) işlem çalışır. İkinci adım (tüketiciyi doğrulama) veri tabanından yalnız okuma yaptığı için onun bir telafi işlemi yoktur; geri dönerken veri tabanında değiştirilecek bir şey kalmaz.

8.4 Saga'nın koordinasyonu

Öğrenciye göre Saga iki türlü koordine edilebilir: orkestrasyon ve koreografi. Orkestrasyonda bir merkez adımları kontrol eder. Telafi geriye doğru yapıldığı için hangi adımların tamamlandığını tutan bir hafıza mekanizması gerekir. Bu bilgi bir SQL veri tabanında, yerel transaction'ın tamamlandığı bilgisini yayan mesaj aracılarında ya da Redis gibi bellek içi bir yapıda tutulabilir. Böylece rollback mekanizması başarılı çalışır. Öğrenci bu iki yöntemin ayrıntısının kendi bölümünde olmadığını söyleyerek sunumu bitirir.

> **Ek bilgi (derste anlatılmadı):** Koreografi tabanlı saga'da merkezi bir koordinatör yoktur; her servis bir olayı dinler, kendi yerel transaction'ını yapar ve yeni bir olay yayınlar. Orkestrasyon tabanlı saga'da ise bir orkestratör nesnesi her adımda hangi servisin ne yapacağını komut mesajlarıyla bildirir ve cevaplara göre sıradaki adıma ya da telafiye geçer.

Hoca sunumdan sonra Saga'nın aslında biraz daha ileri bir konu olduğunu belirtir (00:43:14).

9. Servislere ayrıştırma süreci ve sistem operasyonları

9.1 Problem ve süreç

Üçüncü sunum kitabın ikinci bölümünü anlatır: bir mikroservis mimarisinin nasıl servislere ayrıldığı, nasıl daha iyi bir sistem haline getirildiği ve özellikle CI/CD ve DevOps süreçlerine nasıl uygun hale getirildiği. Kitap bunu ayrıştırma (decomposition) yöntemleri olarak anlatır.

Öğrencinin tanımına göre mikroservis, tek bir sorumluluğu olan, bağımsız, dağıtılabilir ve izlenebilir bir yapıdır (ekrandan, 00:50:00). Mikroservise neden ihtiyaç duyulduğu sorusunun cevabı CI/CD süreçleriyle uyumdur. Bir uygulama büyüdükçe birbirleriyle ilişkili birden fazla servis ortaya çıkar ve bu ilişkilerin iyi tanımlanması gerekir. Servisler bazen farklı bölümlerde geliştirilir. Bu bölümlerin ortak bir noktada buluşması, ama bir bölümün yaptığı değişikliğin diğerini etkilememesi gerekir. Mikroservis mimarisi bu ihtiyaca yönelik gelişmiştir.

Başlangıçtaki büyük problem, servislerin nasıl belirleneceğidir. Sistemin istendiği gibi çalışması için servislerin ve operasyonların doğru tanımlanması gerekir. Kitap bunu iteratif bir süreç olarak tanımlar: belirlenen model sürekli yeniden düzenlenir ve sürece yaratıcılık katılması gerekir.

Süreç sahada başlar. Mikroservis mimarisinin geliştirileceği kuruma gidilir ve uzmanlara sorulur: Bu işi nasıl yapıyorsunuz? Ürün ya da veri nereden gelip nereye gidiyor? Nasıl işliyorsunuz, hangi süreçlerden geçiyor? Öğrenci bunu hastanelerdeki anamnez formuna benzetir: işin hikâyesi alınır. Sonra, derste de yapıldığı gibi, bu hikâye isimler ve fiiller diye ayrılır. İsimler servisler ve sınıflar, fiiller ise operasyonlar olur. Herhangi bir serviste yapılan değişiklikten

sonra bütün modelin nasıl etkilendiği, neyin değişmesi gerektiği ve uyumun korunup korunmadığı yeniden gözden geçirilir.

Kitaptaki örnekte gereksinimler kullanıcı hikâyeleri gibi verilir: bir tüketici sipariş vermek ister; bir restoran siparişleri kabul etmek, onaylamak ya da reddetmek ister. Bunlardan "sipariş oluştur" ve "siparişi kabul et" gibi fonksiyonlar çıkarılır.

Kitap mimariyi kurmak için üç adım tanımlar:

1. Sistem operasyonlarının tanımlanması.
2. Bu operasyonları gerçekleştirecek servislerin tanımlanması (ör. sipariş servisi, restoran servisi).
3. Servislerin birbiriyle ve dışarıyla haberleşmesini sağlayacak API'lerin ve aralarındaki ilişkilerin tanımlanması.

9.2 Sistem operasyonları

Slayttaki başlıklar şunlardır (ekrandan, 00:52:30): sistem operasyonu nedir (komut-sorgu, soyut), neden önemlidir (DevOps), komutlar-sorgular ve alan modeli.

Kitap sistem operasyonlarını komutlar ve sorgular diye ikiye ayırır. CRUD işlemlerinden okuma (read) sorgudur: belirli bir sorgu çalıştırılır ve veri getirilir. Oluşturma, güncelleme ve silme ise komuttur. Sistem operasyonları, bir servisin ihtiyaç duyduğu veriyi diğer servislerden ya da kullanıcıdan alan, bazen işleyen, birbirini çağıran yapılardır.

Bu operasyonların doğru servise doğru tanımlanması neden önemlidir? Çünkü sistem sürekli gelişecektir: bazen bir hata çözülecek, bazen yeni bir özellik eklenecektir. Bir servise eklenen özelliğin diğer servisleri ne kadar etkileyeceği ve onlarda ne kadar değişiklik gerektireceği sorusu her seferinde ortaya çıkar. Kitabın hedefi bu etkiyi en aza indirmektir. Servisler olabildiğince birbirinden ayrıştırılırsa, bir yerde yapılan değişiklik diğerlerini fazla etkilemez, iş yükü azalır ve hızlı, iyi bir sonuç elde edilir.

10. Yüksek seviyeli alan modeli

İlk oluşturulan şey alan modelidir. Alan modeli, hikâyede neyin servis olabileceğini belirlemeye yarar. Bu model ilk haliyle kalmaz; süreç ilerledikçe farklı ihtiyaçlara göre bir alt alan (subdomain) doğrudan ayrı bir servis olarak ortaya çıkabilir. Önemli olan, alan modelinde servisleri oluşturmak ve hangi operasyonlara ihtiyaç olduğunu belirlemektir. Başlangıçta sipariş, restoran ve teslimat gibi servisler varsayılır; "sipariş oluştur" ve "siparişi kabul et" gibi operasyonlar belirlenir.

Kitaptaki "sipariş verme" (Place Order) hikâyesi şudur (ekrandan, 00:55:30):

```
Verilen bir tüketici
Ve bir restoran
Ve o restoran tarafından teslim edilebilecek bir teslimat adresi/zamanı
Ve restoranın sipariş asgari tutarını karşılayan bir sipariş toplamı
Tüketici, restoran için bir sipariş verdiğinde
Tüketicinin kredi kartı onaylanır
Sipariş, PENDING_ACCEPTANCE (KABUL BEKLENİYOR) durumunda oluşturulur
Sipariş, tüketiciyle ilişkilendirilir
Sipariş, restoranla ilişkilendirilir.
```

Öğrenci, başlangıçta yalnız tüketici, restoran, sipariş oluşturma ve kabul etme varken hikâyeye girildikçe gereken operasyonların çeşitlendiğine dikkat çeker. Kredi kartı onayı gibi adımlarla senkron ve asenkron işleyişe de değinilir. Ona göre bu ayrımları yaparken, özellikle alt alanları belirlerken, kesin bir kural yoktur; iş öznel bir yorumdur ve sistemin ait olduğu kurumun ne iş yaptığıyla ilgilidir.

İkinci hikâye siparişin kabulüdür. Sipariş PENDING_ACCEPTANCE durumundadır ve teslim için uygun bir kurye vardır. Restoran siparişi belirli bir saatte hazırlama sözü vererek kabul ederse siparişin durumu ACCEPTED olur, siparişin "promise by" zamanı verilen sözle güncellenir ve bir kurye siparişi teslim etmek üzere atanır. Bu operasyonlar yürütülürken başka servislerin bilgisine de ihtiyaç duyulur; bu bilgiler API'lerle getirilir.

Hikâyelerden çıkarılan sınıflar ve ilişkiler şunlardır (ekrandan, 00:58:30):

```
Order --Placed by--> Consumer
Order --For--> Restaurant
Order --Assigned to--> Courier
Order --Paid using--> PaymentInfo
Order --> DeliveryInfo, OrderLineItem
Consumer --Pays using-- PaymentInfo
Restaurant --> MenuItem, Address
Courier --> Location
OrderLineItem --> MenuItem
DeliveryInfo <>-- Address

Order:         state, ...
Restaurant:    name, ...
Courier:       available, ...
Location:      lat, lon
PaymentInfo:   creditcardId, ...
DeliveryInfo:  deliveryTime
OrderLineItem: quantity
MenuItem:      name, price
Address:       street1, street2, city, state, zip
```

Öğrenci modeli yorumlar. Siparişin bir durumu (state) vardır. Teslimat bilgisi bir adrese ihtiyaç duyar. Kuryenin uygun olup olmadığı (available) kurye sınıfının içine konmuştur. Öğrenciye göre bu bilgi ayrı bir teslimat sınıfına da taşınabilirdi. Bu, alan modelinin tek bir doğru cevabı olmadığını gösterir.

11. İş yeteneklerine göre ayrıştırma

Birinci ayrıştırma yöntemi, hizmetleri iş yeteneklerine (business capability) göre tanımlamaktır. İş yeteneği şirketin ne yaptığını tanımlar (ekrandan, 01:00:00). Yöntem, şirketin işinde önemli olan kavramların belirlenmesini ve bunların servis olarak tanımlanmasını önerir. Ayrıştırmada ana hedef, şirketin ne yaptığını tanımlayan parçalardır.

Kitabın örneği banka çekidir. Eskiden çek yatırmak için kâğıtla bankaya gidilirdi; şimdi ATM'den ya da telefondan yatırılır. İş yeteneği (çek yatırma) değişmemiştir, yalnız nasıl yapıldığı değişmiştir. Bu yöntemle kurulan mimaride değişiklikler genellikle operasyon düzeyinde kalır ve servis aynı kalır. Bu da mimariyi değişime dayanıklı kılar.

FTGO için iş yetenekleri şöyle sıralanır:

- **Tedarikçi yönetimi:** kurye yönetimi (kurye bilgileri, konum, uygunluk) ve restoran bilgi yönetimi (menüler, konumlar, çalışma saatleri).
- **Tüketici yönetimi:** tüketiciyle ilgili bütün bilgiler.
- **Sipariş alma ve yerine getirme:** sipariş yönetimi, restoran sipariş yönetimi ve lojistik (kurye uygunluğu yönetimi ile teslimat yönetimi).
- **Muhasebe:** tüketici muhasebesi, restoran muhasebesi ve kurye muhasebesi.

Öğrenci burada bir ayrıntıya dikkat çeker: kuryenin uygunluk bilgisi alan modelinde kurye sınıfındaydı, burada ise sipariş alma ve yerine getirme altındaki lojistiğe konmuştur. Ayrıca servis sayısı arttıkça birbirinin çıktısına ihtiyaç duyan ya da uyumsuz kalabilen servisler ortaya çıkabilir.

Yetenek hiyerarşisinden servislere geçiş şöyledir (ekrandan, 01:02:45):

```
Tedarikçi yönetimi
  Kurye yönetimi                    -> Courier Service
  Restoran bilgi yönetimi           -> Restaurant Service
Tüketici yönetimi                   -> Consumer Service
Sipariş alma ve yerine getirme
  Sipariş yönetimi                  -> Order Service
  Restoran sipariş bileti yönetimi  -> Kitchen Service
```

```
  Lojistik
    Kurye uygunluğu yönetimi        -> Delivery Service
    Teslimat yönetimi               -> Delivery Service
Muhasebe
  Tüketici / restoran / kurye muhasebesi -> Accounting Service
```

Öğrenci iterasyonun yönünü de gösterir: örneğin tüketici yönetimi ileride iki ayrı servise bölünebilir. Bu servis sayısını artırır ve bir dezavantaj gibi görünebilir. Ama servisler arasındaki bağımlılık azalır ve her servis tek bir işle uğraşır. Mimariyi kurarken ana hedeflerden biri de budur.

Bir hizmetin hangi iş yeteneğine karşılık geldiğini belirlemek öznel bir karardır ve iş süreçlerinin derinlemesine anlaşılmasına dayanır. Kitabın yazarı kendi kararlarını şöyle gerekçelendirir:

- Tedarikçi yönetiminin alt yetenekleri iki servise eşlenmiştir, çünkü restoranlar ve kuryeler çok farklı tedarikçi türleridir.
- Sipariş alma ve yerine getirme yeteneği, sürecin farklı aşamalarından sorumlu üç servise eşlenmiştir.
- Kurye uygunluğu yönetimi ve teslimat yönetimi tek bir servise eşlenmiştir, çünkü bunlar derinlemesine iç içe geçmiştir.
- Ödeme ve faturalama şimdilik aynı servis (Accounting) altında ele alınmıştır.

Öğrenci üçüncü kararı şöyle açıklar: birbirine çok bağımlı iki yetenek ayrı servis olursa birinde yapılan değişiklik diğerini de etkiler ve iki serviste birden değişiklik gerekir. Birleştirildiklerinde değişiklik tek bir servisi etkiler.

12. Alt alan (subdomain) desenine göre ayrıştırma

İkinci yöntem, alt alan desenini uygulayarak servisleri oluşturmaktır. Bu yöntem nesne yönelimli alan modellemesinden, yani Domain-Driven Design'dan (DDD) esinlenir. Asıl hedef, farklı servislerle uğraşan ekipler arasında ortak bir dil kurmak ve iş alanının farklı yönlerini belirlemektir.

Öğrenciye göre bu yöntemin en çarpıcı yanı şudur: önceki yaklaşımda bütün sistem için tek bir alan modelinden söz ediliyordu. Alt alan yönteminde ise her servisin kendine ait bir alan modeli vardır. Böylece her servis yalnız tek bir işle ilgilenir ve servisler arası bağımlılık azalır. Bunu ekip yapısıyla düşünmek kolaydır: 6 ya da 10 kişilik bir ekibin yaptığı değişiklik yalnız kendi servisini etkiler. Bu da hızlı entegrasyon yapılmasını ve uygulamanın sürekli hizmet vermesini sağlar.

DDD'deki sınırlı bağlam (bounded context) kavramı, bir servisin diğerini etkilememesi, aradaki bağın azaltılması ve servislerin birbirinden yalıtılması anlamına gelir. Bunun bir sonucu daha vardır. Bir kavram, farklı servisleri geliştiren ekipler arasında farklı bir şeyi ifade edebilir. Her bağlam ayrı tutulduğunda bir kavram bir ekip içinde hep aynı şeyi ifade eder ve böylece ortak dil kurulmuş olur.

FTGO alanındaki eşleme şöyledir (ekrandan, 01:07:50):

```
FTGO alanı
  Order taking subdomain -> Order Service      (Order domain model)
  Delivery subdomain     -> Delivery Service   (Delivery domain model)
  Kitchen subdomain      -> Kitchen Service    (Kitchen domain model)
  .... subdomain         -> .... Service
  Accounting subdomain   -> Accounting Service (Accounting domain model)
```

Her alt alanın bir alan modeli vardır ve bir servisle ifade edilir. Öğrenciye göre şirketi tanımlayan iş adımları başlangıçta çok büyük ve karmaşık görünür; bu biçimde parçalandığında ise basite iner ve her servis tek bir görevi yerine getiren bir yapıya dönüşür.

13. Ayrıştırma yönergeleri ve engeller

13.1 SRP ve CCP

Slaytta ayrıştırma yönergeleri olarak SRP ve CCP, sonuçlar olarak da uygulama, tanrı sınıfları, sürdürülebilirlik ve ölçeklenebilirlik başlıkları yer alır (ekrandan, 01:10:00). Öğrenciye göre mikroservis tasarımında 11 ilke vardır ve kitap bunlardan yalnız ikisini anlatır.

- **Single Responsibility Principle (SRP, tek sorumluluk ilkesi):** Her servisin tek bir işi olmalıdır. Servis tek bir görevden sorumludur ve yalnız onu gerçekleştirmeye çalışır.
- **Common Closure Principle (CCP, ortak kapanış ilkesi):** Öğrenci bunu, birbirine yakın işlemleri yapan servislerin bir sınıf içinde olması ve bunlardan biri etkilendiğinde ya da verisi güncellendiğinde o gruba ait bütün servislerin birlikte güncellenmesi diye anlatır.

  Ders sonrası not: CCP'nin sunumdaki anlatımı (01:09:26) ilkeyi tam karşılamıyor. İlkenin bilinen ifadesi büyük olasılıkla şudur: aynı nedenle değişen sınıflar aynı pakette toplanmalıdır. Mikroservis bağlamında bu, aynı nedenle değişecek bileşenlerin aynı servise konması demektir; böylece bir gereksinim değişikliği yalnız bir servisi etkiler.

Servisler ve aralarındaki ilişkiler belirlenirken bu iki ilkeye dikkat edilirse baştaki hedefe yaklaşılır: birbirinden yalıtılmış, belirli işleri yapan servislerden oluşan bir mimari.

13.2 Tanrı sınıfları

İlkeleri uygularken çıkan en önemli sorunlardan biri tanrı sınıflarıdır (God classes). Tanrı sınıfı, birçok işi gören yapıların tek bir sınıf altında toplanmasıdır. Bunları ayrı sınıflar olarak tanımlamak daha uygundur. Ama bazen bazı sınıflar ya da servisler birbirine o kadar bağlıdır ki ayırmak çok güçtür. Kitap burada Saga'ların kullanılabileceğini söyler ve bunu sonraki bölümlerde anlatır.

> **Ek bilgi (derste anlatılmadı):** FTGO'daki tipik tanrı sınıfı `Order` sınıfıdır: sipariş, mutfak, teslimat ve muhasebe süreçlerinin hepsi ona dokunur. Kitapta önerilen çözüm büyük olasılıkla DDD yaklaşımıyla her servise `Order`'ın yalnız kendi ihtiyacı olan kısmını taşıyan ayrı bir alan modeli vermektir; örneğin mutfak servisinde aynı kavram `Ticket` olarak, teslimat servisinde `Delivery` olarak modellenir.

13.3 Sürdürülebilirlik ve ölçeklenebilirlik

Ayrıştırma yönergeleri uygulandığında sürdürülebilirlik ve ölçeklenebilirlik sağlanır. Bir sistem bugün üç ya da beş makineyle gereksinimleri karşılıyor olabilir. Ama ileride sistemi büyütmek, ekibi büyütmek ya da yeni bir alan açmak gerekebilir; mimari buna uygun olmalıdır. Öğrenciye göre şimdiye kadar anlatılanların hepsi sürdürülebilirliğe katkı sağlar: her servis ve her bölüm tek bir işten sorumlu olduğu için birinin değişikliği diğerlerini etkilemez, çözüm hızlı üretilir ve sistem kesintisiz hizmet verir.

13.4 Mikroservis mimarisini uygulamanın zorlukları

Slayttaki beş engel şunlardır (ekrandan, 01:15:30):

- **Gecikme (ağ):** Bir API çalışırken başka bir servisin API çıktısına ihtiyaç duyabilir ve onu beklemek zorunda kalır. İstekler dışarıdan da gelebilir, servisler birbirinden de istekte bulunabilir. Çok sayıda servis ağ gecikmesini artırır; bu en aza indirilmeye çalışılır.
- **Tutarlılık (veri):** Bir sipariş bilgisi bazen mutfaktan, bazen teslimat servisinden getirilir; bu bilgilerin her iki tarafta da aynı olması gerekir. Bir servis iki ayrı sunucuda çalışıyorsa, iki sunucu aynı veriyi aynı biçimde tutmalı ve kullanıcıya aynı değeri göstermelidir. Bu, arka taraftaki sunucuların ve veri tabanlarının birbiriyle iyi iletişim kurmasını gerektirir.
- **Kullanım (senkron iletişim):** İletişimin eş zamanlı mı olacağı, yoksa bir operasyonun gerçekleşmesi için dışarıdan bir tetikleyiciye ya da başka bir işlemin bitmesine mi bağlı olacağı tutarlılığı doğrudan etkiler.

- **Karmaşa (dağıtılmış sistem):** Sistem çok sayıda servise bölündüğünde çok sayıda API yazmak gerekir; operasyonlar ve aralarındaki ilişkiler çok karmaşık bir yapıya dönüşebilir. Amaç işi basite indirgemek ve tutarlı kılmaktır.
- **Tanrı sınıfları (ayrıştırma):** Yukarıda anlatıldığı gibi, ayrıştırmayı en çok zorlaştıran engeldir.

Öğrenci gecikme ve tutarlılık sorunlarının tek tek bakıldığında çözülebilir göründüğünü, ama hedefin hepsini birlikte çözebilmek olduğunu söyler. Bu bağlamda dağıtık sistemlerdeki CAP teoremini anar: tutarlılık (consistency), erişilebilirlik (availability) ve bölünme (partition) kavramlarından aynı anda en çok ikisi sağlanabilir, biri her zaman diğerleriyle çakışır.

> **Ders sonrası not:** Öğrenci CAP'in üçüncü harfini "Partition" diye verdi (01:15:21). Kastedilen büyük olasılıkla partition tolerance'tır, yani ağ bölündüğünde sistemin çalışmayı sürdürmesi. Dağıtık sistemde ağ bölünmesi göze alınmak zorunda olduğundan, bölünme anında tutarlılık ile erişilebilirlikten biri feda edilir; "üçünden en çok ikisi" ifadesi bu anlamdadır.

14. Servis API'lerinin tasarımı ve sonuç

14.1 Servis API'leri

Servisler ve operasyonlar belirlendikten sonra sıra servis API'lerinin tasarımına gelir. Slayttaki başlıklar şunlardır (ekrandan, 01:17:05): operasyonların servislere atanması, gerekli API'lerin belirlenmesi, senkron-asenkron iletişim, kendi kendine yeten servisler ve API Gateway kullanımı.

API tasarımı, operasyonların servislere atanmasıyla ilgilidir. Bir operasyon bir servis tarafından sunulduğunda başka servislerin operasyonlarına ihtiyaç duyuyor mu, yoksa dışarıdan gelen bir istekle mi çalışıyor? Bunların belirlenmesi gerekir. Öğrenciye göre burada asenkron iletişime önem verilir. Bir servise API tanımlanırken o servisin hangi servislerin API çıktılarına ihtiyaç duyduğu, bu isteklerin hangileri olduğu ve hangi sırayla gelmesi gerektiği tanımlanmalıdır.

Önemli bir kavram da kendi kendine yeten (self-contained) servislerdir: bir servis tek bir işten sorumlu olmalı ve öyle tasarlanmalıdır. Dışarıdan gelen ya da başka servislerin çıktısına ihtiyaç duyan API'ler için API Gateway kullanılır. API Gateway farklı API'lerin çıktılarını alır, birleştirir ve tek bir çıktı olarak sunar; bir birleştirici gibi düşünülebilir.

14.2 Sonuç ve değerlendirme

Sonuç slaytı avantajlar, dezavantajlar (senkron-asenkron), trendler, operasyon-servis doğru ataması, düşük bağlam (servisler) ve API Gateway (basit veri entegrasyonu) başlıklarını içerir (ekrandan, 01:20:00).

- **Avantajlar:** DevOps sürecine önemli katkı sağlanır. Her servis tek bir hizmetten sorumludur. Operasyonlar, servisler ve API'ler ölçeklenebilir ve sürdürülebilir bir yapıya uygun tasarlanır.
- **Dezavantajlar:** Senkron ve asenkron işlemlerin ayrılması ya da tanrı sınıflarının parçalanması her zaman mümkün olmayabilir. Bunun için kullanılan farklı yöntemler kitabın sonraki bölümlerinde anlatılır.
- **Doğru atama:** Operasyonların servislere doğru atanması çok önemlidir. Süreç, sürekli yenilenen ve düzenlenen bir model üzerinde iteratif olarak ilerler.
- **Düşük bağlam:** Her servis yalnız kendisinden istenenlerden sorumlu olacak biçimde tasarlanmalıdır.
- **API Gateway:** Farklı servislerin çıktılarını ve dışarıdan gelen istekleri birleştirerek tek sonuç olarak sunan basit bir veri entegrasyonu sağlar.

Öğrenci kitabın ikinci bölümünü şöyle toparlar: bir mikroservis mimarisi kurulurken önce operasyonlar ve servisler belirlenir ve yüksek seviyeli bir alan modeli kurulur. Sonra iki yöntemden biri kullanılır: iş yeteneklerine göre ya da alt alan modellerine göre ayrıştırma. Ayrıştırmada SRP ve CCP ilkeleri gözetilir. En son bu servislerin API'leri tasarlanır ve böylece başlangıç olarak minimal bir mikroservis mimarisi ortaya çıkar.

Dersin Toparlaması

Bu derste üç sunum, mikroservis mimarisini üç farklı açıdan ele aldı. Birinci sunum mikroservisin ne olduğunu, bağımsızlık ve esneklik gibi temel özelliklerini, 4+1 görünüm modelini, katmanlı ve altıgen mimariyi ve FTGO örneğini tanıttı. Üçüncü sunum, bir sistemin servislere nasıl ayrılacağını gösterdi: sistem operasyonlarından ve alan modelinden başlayıp iş yeteneklerine ya da DDD alt alanlarına göre servisler çıkarmak, SRP ve CCP ilkelerini gözetmek, gecikme, tutarlılık ve tanrı sınıfları gibi engelleri tanımak ve en sonunda servis API'lerini tasarlamak.

İkinci sunum ise bu ayrıştırmanın doğurduğu en önemli sorunlardan birine odaklandı: her servisin kendi veri tabanı olduğunda birden fazla servise yayılan bir işlemin tutarlılığı nasıl korunur? Monolitte `@Transactional` ile tek bir rollback yetiyordu. Mikroserviste senkron ve kırılgan Two-Phase Commit yerine, yerel transaction'ları mesajlarla zincirleyen ve başarısızlıkta ters sırayla telafi işlemleri çalıştıran Saga deseni kullanılır. Hocanın da belirttiği gibi Saga, ayrıştırma konusunun üzerine kurulan daha ileri bir konudur.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.